

Multiclass vulnerability and clone detection in Ethereum smart contracts using Block-wise Abstract Syntax Tree based Federated Graph Neural Networks

Shruti Sharma , Ankur Ratmele ^{*}, Abhay Deep Seth

Department of Computer Engineering, STME, SVKM's NMIMS, Indore, 453112, Madhya Pradesh, India

ARTICLE INFO

Keywords:

Abstract syntax tree
Ethereum smart contracts
Federated graph neural network
Vulnerability and clone detection

ABSTRACT

Smart contracts on blockchain networks autonomously execute applications based on predefined conditions, making their security-critical due to the potential for significant financial losses from vulnerabilities. Current vulnerability detection algorithms commonly rely on expert-defined rules, which are prone to errors and insufficient for identifying complex vulnerability patterns. Given the immutability of smart contracts post-deployment, ensuring security before deployment is essential. This research presents Block-wise Abstract Syntax Tree based Federated Graph Neural Networks (BAST-FeGNN), a novel approach combining block-wise abstract syntax tree and Federated Graph Neural Networks (FeGNN) to detect code clones and multiclass vulnerabilities in Ethereum smart contracts. The BAST-FeGNN method operates in three stages: it first extracts security-related patterns from the base code using an abstract syntax tree; then, it constructs and normalizes a contract graph using FeGNN to capture critical nodes, analyze data and control flows. This integration of graph-based feature extraction with pattern matching allows precise detection of vulnerabilities like access control issues, reentrancy, and unchecked calls, as well as identifying code clones. Finally, the method pools these features for comprehensive vulnerability detection. BAST-FeGNN significantly enhances vulnerability detection accuracy and scalability, outperforming existing models with an accuracy of 95.35%, recall of 95.58%, F1-score of 95.80%, and precision of 96.10%, making it a robust solution for securing blockchain applications.

1. Introduction

Of late, the blockchain idea has achieved considerable importance. The blockchain software works as a decentralized peer-to-peer system, maintaining a completely desegregated public database completely secured by cryptography. There are so many applications throwing light on message exchange, digital currency transfer, and data storage. In its infancy, bitcoin was the first type of application in blockchain technology. It was rather Ethereum that came up with the name of a new and revolutionary style of contract called a smart contract [1]–[2], changing the scenario. On the other hand, Ethereum's blockchain hosts large numbers of distributed programs implemented as Smart Contracts (SC), making it the first-choice destination for smart contract execution. Smart contracts are computer programs that the Ethereum Virtual Machine (EVM) bytecodes execute and store permanently into the Ethereum blockchain [3]. This decentralization allows inspection and examination of the EVM bytecode of SCs. However, this is also where the threat lies, since once the SC is put up, destructive parts do not have to leave the protection of the source code

^{*} Corresponding author.

E-mail addresses: shruti.sharma@nmims.edu (S. Sharma), ankur22ratmele@gmail.com (A. Ratmele), abhays@iitbhilai.ac.in (A.D. Seth).

anymore because it is public to everyone else. It is obvious to see how transparency could attract the attention of adversaries [4]. Consequently, attacks on weak SCs lead to substantial losses, especially when dealing with valuable assets.

A contract account, distinct from an Externally Owned Account (EOA), lacks a private key and contains a smart contract code. It is supervised by the smart contract code that created it. On Ethereum, a smart contract operates deterministically, and immutably, and is deployed on the blockchain, functioning within the EVM. The inability to modify or remove smart contract codes after deployment fosters trustworthiness, resembling other data distributed on a standard blockchain [5]. This ensures consistent results when executing written functions. SCs are self-regulating and executed via the peers in the meshwork upon satisfying the contract's predefined conditions [6]. To invoke a smart contract, a unique contract address is used, enabling it to perform its operations. Transactions, represented by signed messages from an EOA, invoke the SC while the transaction's destination is the contract address. Consequently, the smart contract operates within the EVM, making use of transactions and their associated data as input [7]–[8]. The transaction information specifies the particular contract function to be executed, along with the corresponding arguments [9].

Recently, Artificial Intelligence (AI) has transformed the software security spectrum, especially in the area of vulnerability detection and code analysis. Deep Learning (DL) that uses the Graph Neural Network (GNN) for example has exceptional in comprehending deep structures like the Abstract Syntax Trees (AST). In addition, the rise of federated learning models also means that models are built to gain knowledge from user data without transmitting it from the user's device. Also, AI has enhanced due to new capabilities of transfer learning and multimodal learning that have made it easier to tackle different vulnerability types in software, making it possible to combat both old and emerging threats in software. These developments are critical to overcoming the security threats associated with SC in blockchain, allowing the design of better and advanced stealth systems [10].

In recent times, an increasing number of vulnerabilities in SCs is discovered, and malicious actors have exploited these weaknesses to attack blockchain systems like Ethereum, resulting in significant economic losses [11]. Hence, it becomes imperative to proactively identify potential issues in SCs and develop more secure alternatives. New and cutting-edge techniques for assessing the security of SCs is created as blockchain security events become more prominent. Only a small number of these methods have merged cutting-edge technology like Machine Learning (ML); the majority still relies on conventional static or dynamic analytic techniques [12,13]. Some ML-based systems designed to recognize SC vulnerabilities require substantial time and effort for manual feature extraction.

Due to the substantial financial losses resulting from vulnerabilities, SC security has attracted significant attention. Current methods for recognizing SC vulnerabilities rely heavily on expert-defined rules or patterns. However, these expert guidelines are error-prone and struggle to cover complex patterns. As a result, existing techniques commonly exhibits low detection accuracy, high false-negative and false-positive rates, and extended processing times. To address these limitations, proposed solutions aim to identify vulnerabilities in both known and unknown patterns using expanded features, thereby enhancing scalability and reducing training time. The key contributions of the research work lie in its development of a BAST-FeGNN method for multiclass vulnerability detection in Ethereum SCs.

- **Integrated model for dual detection:** BAST-FeGNN uniquely integrates both vulnerability detection and code clone identification into a single cohesive system. Unlike traditional methods which handles these tasks separately, this unified approach enables simultaneous identification of a wider range of vulnerabilities and code clones, offering a more comprehensive solution to smart contract analysis.
- **Innovative graph representation:** The method introduces an advanced graph-based representation using an Augmented Abstract Syntax Tree (AAST), which captures complex structural and semantic relationships within the code. When combined with the FeGNN architecture, this sophisticated representation enhances the detection of intricate vulnerabilities and subtle code clones, especially in large-scale smart contracts.
- **Fusion of security and graph features:** BAST-FeGNN improves vulnerability prediction accuracy by merging security pattern features extracted during the security analysis phase with graph-based features from the graph construction stage. This fusion not only boosts detection effectiveness but also tackles the scalability challenges commonly faced by existing techniques, making it highly suitable for extensive smart contract datasets.
- **Advanced feature extraction for precision:** The method employs cutting-edge feature extraction and normalization techniques to enhance the accuracy of detecting diverse types of vulnerabilities and code clones. This approach significantly reduces false positives and negatives, offering a more robust and scalable solution compared to previous systems, ensuring higher reliability for SC vulnerability analysis.

The organization of this research paper is designed as follows: division 2 demonstrates the earliest techniques based on the Ethereum smart contract. Section 3 outlines the proposed methodology and its implementation process. The execution measures of the introduced method are evaluated in Section 4. Lastly, Section 5 brings the paper to a close and outline the findings of the proposed scheme.

2. Literature survey

Different research works have already existed in the writings which are based on the vulnerability detection. DL techniques researched as one of the most effectual and active ways to find smart contract vulnerabilities. Since some vulnerable programmed codes is found in SCs, a smart contract has the biggest dangers to the Ethereum blockchain.

Liu et al. [14] presented a detection of Smart Contract Vulnerabilities (SCV) utilizing combined GNN. The suggested method analyses 300,000 smart contract functions and identifies the vulnerability in function level. Then normalizing graphs were given

to the propagating network for learning the features of graph. At the same time, the security features are removed from the base code utilizing the GNN. To end with, the graph features and extracted pattern features were combined to get the detected vulnerability outcomes. Hwang et al. [15] presented a detection of SCV using a Code-targeted Network (CodeNet). The suggested method uses code-targeted Convolutional Neural Network (CNN) called CodeNet. The suggested method detected the smart contract vulnerabilities lacking strides whereas maintains the context and semantics of smart contract.

Huang et al. [16] has presented multi-task learning based detection of SCV that uses CNN for classifying every task branch. Zhang et al. [17] presented a graph based SCV detection. Abstract Semantic Graph (ASG) based vulnerability detection was suggested for detecting the vulnerability of bytecode and source code. The suggested method detects the vulnerability of bytecode and source code with high false alarm and missed alarm rates

Qian et al. [18] had introduced a Bidirectional Long Short-Term Memory (Bi-LSTM) Known for its capability to manage long-term dependencies effectively due to its gated architecture, making it ideal for tasks requiring extensive contextual understanding. Their approach involved transforming opcodes into feature matrices for multi-label categorization, leveraging Bi-LSTM's strength in handling complex sequential data. Better suited for applications with complex, lengthy input sequences requiring detailed contextual analysis, such as identifying subtle patterns in opcode sequences.

Wu et al. [19] presented a hybrid attention based SCV detection. Hence, a Hybrid Attention Mechanism (HAM) approach was subsequently suggested to find security flaws in SCs. The suggested method does not detect multiclass vulnerability. For a more scalable model of detecting smart contract vulnerabilities, Zhang et al. [20] suggested the Multiple-Objective Detection Neural Network (MODNN) is able to validate 12 categories of vulnerabilities, comprising ten known risks, and find other unknown risks without the requirement for specialized or predetermined knowledge.

Li et al. [21] developed StateGuard, a DL-based model designed to identify state deviation flaws of SCs. StateGuard constructed an AST of the SC, abstracting significant traits for graph modeling. A Graph Convolutional Network (GCN) was then leveraged to reveal imperfections.

Jain et al. [22] utilized Bi-GRU to combine forward and backward sequence information for long-term dependencies. Bi-GRU, while similar to Bi-LSTM, is computationally more efficient due to fewer parameters and simpler gating mechanisms, making it suitable for scenarios where speed and resource optimization are critical. Preferred in environments where resource constraints exist or faster training and inference are prioritized, without significant compromise on performance, such as lightweight smart contract analysis frameworks.

Zhuang et al. [23] proposed a GNNs for SC vulnerability identification. The temporal message propagation network and degree-free graph convolutional neural meshwork was introduced for learning from normalized graphs to effectively detect vulnerabilities. Extensive experiments show the superiority of their approach over existing methods in detecting various vulnerability types. Table 1 analyzed available techniques, highlighting their strengths and weaknesses.

In 2024, Jeon et al. [24] introduced Smart Contract Detection (SmartConDetect), the first static analysis technique for detecting security vulnerabilities in Solidity SCs. SmartConDetect worked on extracting code fragments from Solidity SCs and used a pre-trained Bidirectional Encoder Representation from Transformers (BERT) model to identify vulnerable code patterns.

In 2024, Luo et al. [25] introduced Smart Contract Vulnerability Hunter (SCVHunter), an extensible framework for smart contract vulnerability detection. Specifically, SCVHunter designed a heterogeneous semantic graph construction phase based on intermediate representations and a vulnerability detection phase utilizing a heterogeneous graph attention network for SCs.

In 2023, Tang et al. [26] proposed a lightning cat as a remedied application driven by DL techniques. Three DL models for the vulnerability detection in SCs include optimized code BERT, optimized-Long Short-Term Memory (LSTM), and optimized-CNN. Experimental results showed that under the proposed lightning cat framework, the optimized-code BERT model outperformed the other methods.

In 2024, Colin et al. [27] proposed a vulnerability detection algorithm using a Multi-Layer Perceptron (MLP) and a combined MLP-Long Short-Term Memory (MLP+LSTM) model, trained on opcode and feature vectors.

In 2024, Upadhyaya et al. [28] introduced QuadraCode AI, a multimodal transformer architecture leveraging four distinct modalities. Using cross-attention and concatenation across 12 multimodal frameworks, it improved vulnerability detection in smart contracts compared to traditional unimodal methods.

In 2024, Zhen et al. [29] introduced a model for identifying intelligent contract vulnerabilities using a Dual Attention Graph Neural Network (DA-GNN), which transformed operation code sequences into a feature matrix and utilized a dual attention mechanism for node embedding updates and vulnerability classification.

In 2024, Duy et al. [30] had introduced a VulnSense framework, a multimodal learning approach for efficiently detecting vulnerabilities in Ethereum smart contracts. It integrates features from source code, opcode sequences, and Control Flow Graphs (CFGs) extracted from bytecode. Using BERT, BiLSTM, and GNN models, the framework analyzes these features, with a fully connected layer predicting vulnerabilities.

In 2024, Tong et al. [31] had introduced a multi-label vulnerability classification mechanism leveraging language models like SecBERT, enriched with auxiliary features from techniques like TF-IDF. A trustworthy neural network with an additional output neuron ensures decision reliability and reduces misclassification rates, enabling accurate detection of implicit and explicit vulnerability features.

In 2023, Ma et al. [32] proposed a Hierarchical Graph Attention Trees (HGAT) network-based detection model to improve smart contract vulnerability detection. Functions were abstracted into code graphs using AST and control flow graphs, with node features extracted via a graph attention mechanism to enhance detection capabilities.

Table 1

Analyzing available techniques, highlighting their strengths and weaknesses.

Authors	Techniques	Strengths	Limitations
Liu et al. [14]	GNN	❖ Improved detection accuracy. ❖ Reduces the need for labor-intensive manual analysis	❖ Bytecode-only contracts limit current applicability. Timestamp dependence accuracy is 89.02%.
Hwang et al. [15]	CodeNNet	❖ Enhanced detection performance. ❖ Reducing likelihood of false detection alarms	❖ Detection accuracy is 98.79%; more diverse testing needed for generalization.
Huang et al. [16]	CNN	❖ Reduced computational and storage costs compared to single-task models.	❖ Precision is 82.17%. Optimization of the multi-task learning architecture is necessary.
Zhang et al. [17]	ASG	❖ Enhanced detection accuracy. ❖ Cover a broad range of smart contract formats, improving robustness.	❖ Limited vulnerability scope. ❖ Timestamp dependence accuracy is 79.28%.
Qian et al. [18]	Bi-LSTM	❖ Improved data processing. ❖ Enhanced detection capabilities.	❖ Accuracy is 85%. Limiting the depth of vulnerability analysis.
Li et al. [21]	StateGuard	❖ Increased capability to identify complex state defects.	❖ Obtained precision is 92.24%. Future work needed to refine representations and expertise.
Jain et al. [22]	Bi-GRU	❖ Enhanced accuracy of vulnerability detection.	❖ Accuracy is 96.1%. Requires substantial computational resources.
Jeon et al. [24]	BERT	❖ High speed and efficiency. High precision in detecting vulnerabilities.	❖ Limited attack detection scope. ❖ High memory usage of 10.9 GB.
Luo et al. [25]	SCVHunter	❖ This user-driven aspect enhances focus on critical areas, boosting efficiency.	❖ Requires substantial labeled data; limits detection of new vulnerabilities.
Tang et al. [26]	DL	❖ DL adapts to new vulnerabilities, improving detection accuracy.	❖ Inconsistent performance across different vulnerability types.
Colin et al. [27]	MLP+LSTM	❖ Reduces dimensionality and eliminates contract-specific hexadecimal values.	❖ Vulnerability space scaling needs further improvement. Accuracy is 41.89%.
Upadhyay et al. [28]	QuadraCode AI	❖ Explores 12 multimodal combinations for optimized performance. ❖ Improves reliability of blockchain-based systems through detection	❖ High computational demands for multimodal analysis. Current framework limits integration of all modalities. ❖ Accuracy is 82
Zhen et al. [29]	DA-GNN	❖ ding118 Innovative opcode categorization. ❖ Robust vulnerability classification. ❖ Offers pathways for deeper semantic analysis	❖ Limited to certain vulnerabilities. ❖ Vulnerable to inaccuracies in opcode extraction and representation. As contracts grow, performance vary unpredictably.
Ma et al. [32]	HGAT	❖ Extracts features at multiple levels. ❖ Demonstrates superior reentrancy. ❖ Minimizes overfitting risk by reducing nodes in contract graph.	❖ Face challenges in scaling with increasing contract complexity. ❖ Model updates is necessary as smart contract patterns evolve.
Duy et al. [30]	VulnSense framework integrating BERT, BiLSTM, and GNN for multimodal learning	❖ Combines diverse features (source code, opcodes, CFG) for comprehensive vulnerability analysis	❖ Performance depends on the quality of bytecode and CFG generation
Tong et al. [31]	Multi-label vulnerability classification mechanism leveraging SecBERT, TF-IDF, and trustworthy neural networks	❖ Reduces misclassification rates and improves decision reliability through auxiliary features and trust model	❖ Potentially limited by the scalability of auxiliary feature integration and neuron-based decision metrics

Problem Statement:

Current techniques for detecting vulnerabilities in smart contracts such as CodeNet [15] and StateGuard [21] suffer some drawbacks concerning their capability in capturing intricate structural codes, which often require heavy computing resources and leads to inefficiencies. While techniques like GNNs [14], CNNs [16], and Bi-LSTM [18] networks have shown accurate detection improvements, they are limited in applicability to bytecode alone, and mostly fall prey to false alarms. The BAST-FeGNN system promises mitigation of these constraints by fusing ASTs into GNNs. This innovative mixture enhances the efficacy, scalability, and robustness of detection capabilities against both vulnerabilities. Several experiments have been performed on different datasets. BAST-FeGNN has been found to outperform other existing methods in precision, recall, and F1-score improvement with a significant reduction in false positives and false negatives compared to leading models. Additionally, BAST-FeGNN exhibits strong generalization

capabilities, enabling accurate identification of vulnerabilities in unseen smart contracts, thereby enhancing security for Ethereum-based applications. This comprehensive analysis underscores the model's distinctive contributions and effectively addresses the issues regarding the existing literature, showcasing the relevance and superiority of BAST-FeGNN in the field.

3. Proposed methodology for multiclass vulnerability detection in Ethereum smart contracts

SC based on blockchain are computer applications that only perform when specific needs are met. Due to their immutability once positioned, SCs cannot be modified to rectify errors in the code once SCs are live. The primary difficulty in smart contract security is the effective and timely discover of vulnerabilities.

Software engineering pattern of mistake like code copying pose a chance to the integrity and sanctuary of the project if mistakes are ignored before deployment. This research uses BAST-FeGNN for copying and multiclass vulnerability identification in Ethereum SCs. The work flow of the introduced system is shown in Fig. 1. This approach involves three main stages:

Security Pattern Extraction: In this stage, the method extracts security patterns associated with vulnerabilities from the contract's source code. This is achieved using an AAST. The researchers identify expert patterns that indicate potential security issues. These patterns serve as a basis for detecting vulnerabilities.

Contract Graph Construction and Normalization: The next step involves creating a contract graph from the smart covenant base code. This graph is designed to capture efficient nodes and to represent the data flow and control flow logic embedded within the code. To enhance the accuracy and effectiveness of this graph, the researchers introduce the FeGNN normalization technique. FeGNN helps refine significant features from the graph while ensuring that the semantics of data flow and control flow are maintained.

Vulnerability Detection: In the final stage, the method detects vulnerabilities within the smart contract. This is done by togethering the characteristics separated from the security patterns and the contract graph. By pooling these features together, the method is able to identify various vulnerabilities and cloned sections of code. This is the step which takes on that real taunting challenge of vulnerability detection in smart contracts security. In this vulnerability detection phase, combination occurs between pattern features and graph features in order to, at the end, produce a detection outcome. The method proposed does code clone detections and 6 types of vulnerabilities such as access-control, arithmetic, other, reentrancy, safe and unchecked-calls.

Overall, the solution proposed becomes real in identifying the risks in applications developed on the lines of blockchain. In fact, it draws the relationship between security patterns-the depth of contract graph analysis and the power of FeGNN-and, in that sense, strengthens Ethereum SC's security and integrity. This workflow is designed to ensure that vulnerabilities are identified prior to deployment, minimizing potential security breaches in blockchain applications.

3.1. Data collection

The data is controlled from the datasets which is related to Ethereum SCs that audited using the Slither tool, it contains information about the audited contracts, their vulnerabilities, and the corresponding fixes [33,34]. The subset of big multilabel data packages is taken for this research work. These data are not suitable for accurate detection process so that the preprocessing approach is taken to smooth away the source code present in the dataset.

3.2. Preprocessing of source code collected from dataset

The unstructured source code present in the datasets is initially preprocessed. Preprocessing of source code in Ethereum SCs involves preparing the code for compilation and deployment on the Ethereum blockchain. This process commonly includes several important steps to ensure the code is optimized, secure, and functional within the Ethereum ecosystem. Some common preprocessing steps for Ethereum smart contract source code is given as follows,

Comments Removal: Comments are useful for developers but are not needed in the final deployed contract. Preprocessing involves removing all comments to reduce the contract's size and deployment cost. Comments are stripped using regular expressions or dedicated parsing tools.

Reduces code: Removes redundant whitespaces, comments, and unnecessary code elements to reduce the size of the contract. A small contract will cost much cheaper for deployment on the blockchain.

Error Checking: Basic preprocessing, with automated tools, provides error checking. This catches other issues before the contract is deployed, to reduce the chances of encountering runtime errors on the blockchain.

Version pragma: Ethereum smart contracts generally contain version pragma at source code start. This pragma indicates the compiler version that compiles the contract. This entire preprocessing, compilation process ensures that the smart contract becomes efficient, secure, and functional within the Ethereum network.

The preprocessing steps address unstructured and noisy data. Remove comments, extraneous spaces, and superfluous code elements to minimize deployment size and improve detection accuracy. Code minimization and normalization are utilized to introduce consistency, filter out syntactic noise, and get rid of possible outliers. This preprocessing makes sure that the source code is free from irrelevant or noisy components before applying detection techniques, resulting in a cleaner and more structured dataset that improves the overall performance and stability of the model. Then the preprocessed data can be used for further detection approaches.

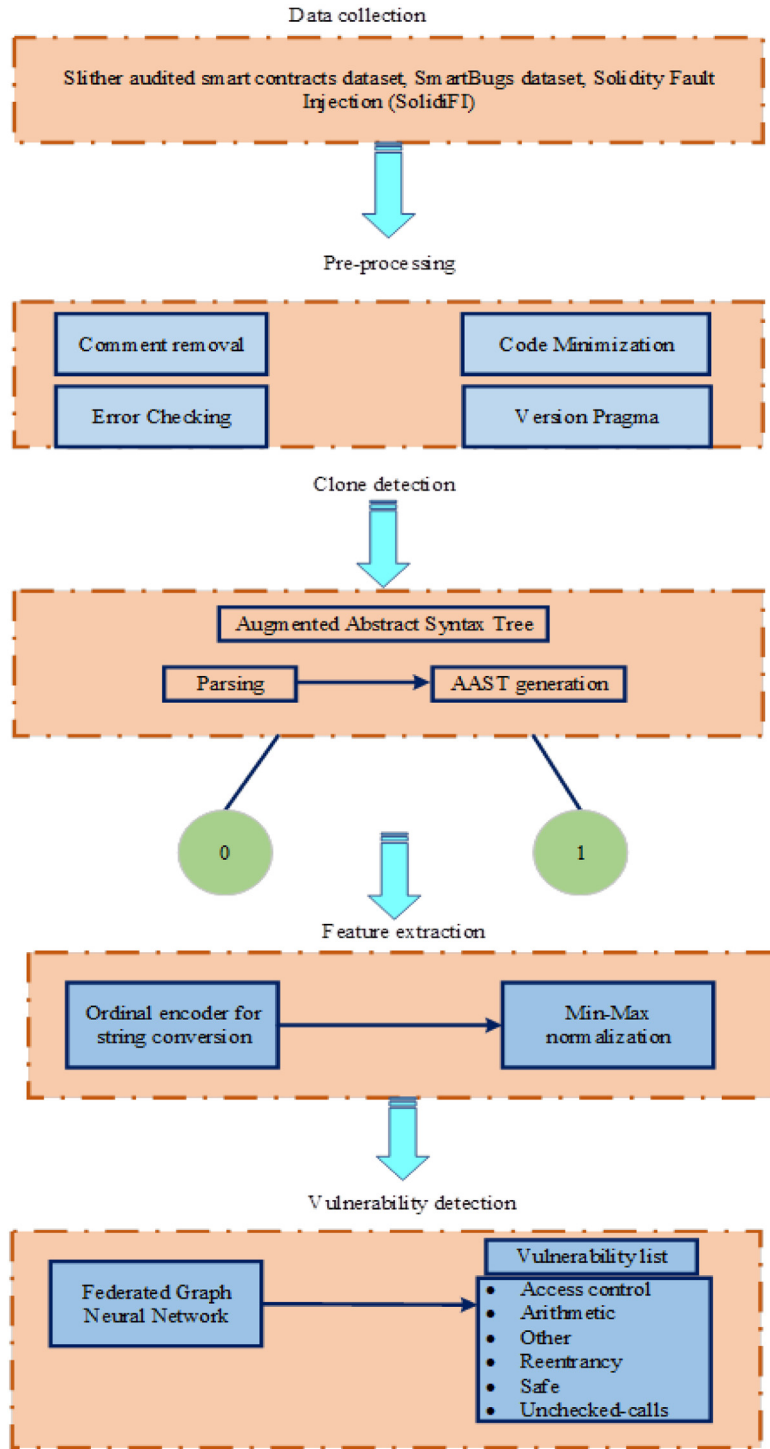


Fig. 1. Architecture of the proposed system for detecting vulnerabilities and code clones using deep learning techniques.

3.3. Efficient features extraction and clone code detection with AAST

Following pre-processing, the source code now clusters without any error or diselement, then proceeds to further processing for AAST generation. Basic Abstract Syntax Trees represent the structure of code, but do not convey any info on the deeper intent of

that code. Augmented AST incorporates contextual understanding; this reflects the purpose of various components of code. Source Code at the first stage of feature extraction and clone identification is passed through parsing, resulting in AAST. This is crucial for identifying features and clones that have different structures but similar functionalities.

3.3.1. Clone detection steps for detecting duplicated codes

The clone detection process identifies instances of code duplication. The initial algorithm focuses on detecting clones at the subtree level, effectively uncovering exact code clones. However, challenges arise in detecting near-identical clones due to complete subtree hashing, which segregates slightly modified trees into separate buckets. To address this, a less precise hash function is utilized, retaining crucial characteristics for near-miss clone detection. Near-miss clones typically result from copy-paste actions with minor modifications, primarily affecting small sub-trees. Thus, a hash function that disregards these small sub-trees proves advantageous. In experiments, a hash function that ignores identifier names is employed, facilitating the grouping of similar trees for accurate differentiation. Rather than exact equality, the focus is on similarity, determined by user-defined parameters and calculated using Eq. (1),

$$S_{\text{simi}} = \frac{2 \times S}{2 \times S + L + R} \quad (1)$$

where, S_{simi} is the similarity between two subtrees, S is the various of shared nodes, L is the number of various nodes in subtree-1 and R is the various nodes in sub tree-2. Subsequently, each pair of sub-trees within the identical hash bucket is assessed. If their correlation surpasses the defined threshold, the pair is included in the clone list, leading to the elimination of corresponding sub-clones.

Clone pair consists of two identical clones, while a clone class known as a clone cluster or clone group, comprises multiple identical clones. Clones are categorized based on fixed or free granularity. In the case of free granularity clone detection, syntactic boundaries like functions, blocks, or sentences are not used, and the entire source code is considered. A constant label c_{ij} is set for the two code remains X_i and X_j to signify whether X_i and X_j is a clone pair. After, create a training set $D_{\text{set}} = \{X_i, X_j, c_{ij}\}$ for a set of code pairs with known clone labels [35]. Training a network involves teaching it to learn a function $\delta()$ that map a given code segment to a corresponding feature vector, in a way that ensures that similarity scores between any two code fragments is calculated that are $c_{ij} = \delta(X_i, X_j)$ as close to each other as possible. During the inference phase, establish a threshold value that distinguishes between false and true clone pairings in pairs of code. This threshold aids in identifying clone pairs effectively. The clones involve the addition, modification, or removal of statements while maintaining the general structure of the code.

Despite differences in syntactical structures, the overall functionality and purpose of code fragments usually remain similar. Code clones shown as fragments with varied syntax that perform the same function, necessitating an analysis of underlying logic rather than mere textual comparison. Although visually distinct, these fragments exhibit functional equivalence through their shared outcomes. For clone detection, the FeGNN model processes pairs of code snippet features and outputs a binary classification indicating whether features are clones or non-clones. The model employs the categorical focal cross entropy loss function to achieve this binary classification, optimizing performance by focusing on challenging examples.

3.3.2. Feature extraction from generated AAST

The processing of a code fragment begins by parsing it into its AAST. Subsequently, a graph representation of the AAST for the code section is constructed by incorporating edges to represent control and data flow. Then initialize the edges and nodes of AAST. These vectorized AASTs are jointly fed into a FeGNN, which calculates vector representations for all nodes in both AASTs. By pooling the node vectors and constructing a graph-level vector forms for every AAST individually, a readout function is used to discover clone codes [35]. By the vector demonstrations, their similarity is measured using the cosine correlation of the two vectors. The 2 code remains are regarded as to be a clone pair if the similarity score is more than the criterion. The structure is configured with categorical focal cross entropy loss function which improves learning efficiency. Improved neural network predictions would result from reducing the cross-entropy among the true distribution of the classes and the distribution modeled by the FeGNN network.

The proposed method enhances code clone detection by parsing code into AAST and constructing detailed graph representations with multiple edge types. The FeGNN model, utilizing categorical focal cross entropy loss, processes these representations to classify code snippets based on functional equivalence, even with syntactical differences. The incorporation of ordinal encoding and min-max normalization optimizes feature processing, improving the model's accuracy and efficiency in detecting clones and vulnerabilities in diverse codebases. The categorical focal cross entropy loss function $\text{Focal}_L()$ is described in Eq. (2),

$$\text{Focal}_L(y, \hat{y}) = -\alpha(1 - \hat{y})^\beta \text{CE}(y, \hat{y}) \quad (2)$$

where, each class weight is represented as α , and categorical cross-entropy function is indicated as CE and β is the parameter that causes the loss to grow for correctly identified examples with low confidence. The dimension of original and predicted value is represented as y and $\hat{y}$, respectively. In the clone detection task, the proposed model aims to predict a solitary actual value representing the resemblance amongst two code snippets, denoted as d . For the construction of graph representations for programs, the following edges are taken into consideration:

- *Child*: In accordance with the AST structure, this edge links an AAST node that is not terminal to each of its children.
- *Parent*: This edge connects a non-root AAST node to its parent node within the AST hierarchy.
- *Next Sibling (NextSib)*: To account for the graph neural network's inability to consider node order, a *NextSib* edge is established to connecting a node to its next sibling from leftward to right.

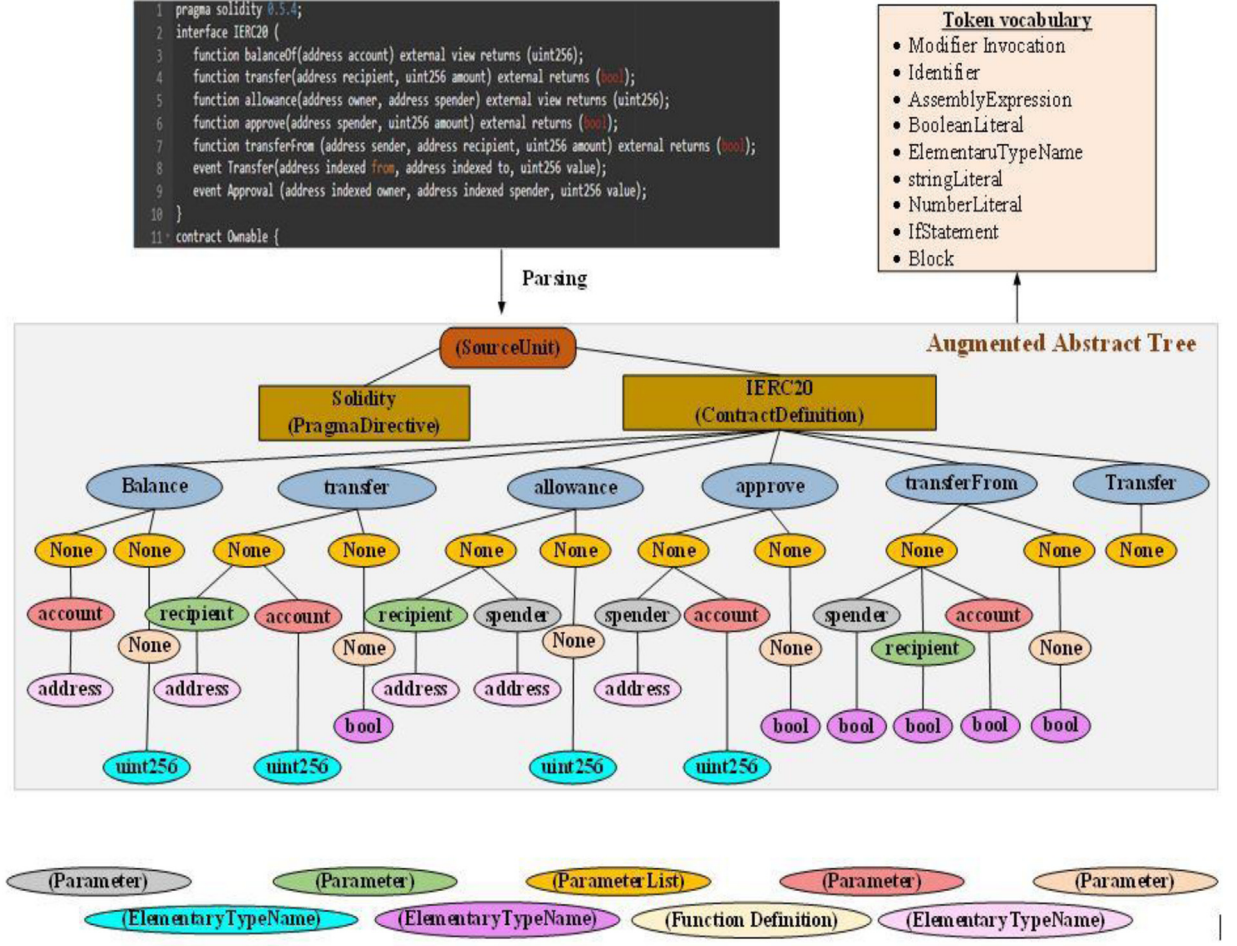


Fig. 2. Feature extraction from generated Augmented Abstract Syntax Tree.

- *NextToken*: Within ASTs, terminal nodes correspond to the tokens representing base code identifiers.
- A terminal node and the subsequent source code token are connected by a *NextToken* edge.
- *NextUse*: A node of variable usage is connected to its later appearance by a *NextUse* edge. Utilize *NextUse* edges to capture useful AST-derived data flow knowledge. Totally 33 features are extracted from AAST as shown in Fig. 2.

By incorporating these types of edges into the graph representation, the proposed model adeptly comprehends the connections and configurations within code snippets for accurate clone detection.

Ordinal encoder for string conversion: An ordinal encoder is a data preprocessing technique used in data analysis to convert categorical string data into numerical values. It is particularly useful when working with numeric input, as it never directly handles text data.

Let us denote the set of unique categories in a categorical feature C as $C_1, C_2, \dots, C_k$, where k is the number of unique categories. For each value a in the categorical feature C , apply the mapping function f , $a_{\text{encoded}} = f(a)$, where $f(a)$ is the encoded numerical value.

The feature extraction column contains categorical string columns. These columns represent features that are assigned a unique integer value to each category, which is listed in Table 2.

Into the Table 2, feature here has each mapping of integer values that maintain the ordinal relationship. The encoding makes it possible for numerical representation to mirror the ordering or ranking of the categorical features where applicable. The ordinal values replace the original categorical data in the features, making the derivation of data simpler for the input into the FeGNN algorithm. Thereafter, the numeric feature values undergo normalization processes. These are all done to ensure that the data end up in a common range and distribution. This is a key step in upholding the integrity of feature values so that processing and subsequent analysis by FeGNN are more efficient and effective. Such a preprocessing approach allows FeGNN to process the data better with respect to ordinal relationships between categorical variables and, thus, improve the performance and accuracy of models.

Table 2
Ordinal encoder for string conversion.

Features	Unique integer value
Assembly Expression	0
Block	1
Boolean Literal	2
Elementary TypeName	3
Identifier	4
If Statement	5
Modifier Invocation	6
Number Literal	7
String Literal	8

Min-max normalization Technique: In the context of vulnerability identification for smart Ethereum contracts, data normalization is applied to the features separated from the contract code before training the FeGNN model. If the features separated from the contract code have numerical values, normalization scale these values to a standard range, such as [0,1]. For a given feature x with minimum value $x_{\min}$ and maximum value $x_{\max}$, the normalized value x' within the range [0, 1] is calculated by Eq. (3).

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (3)$$

This prevents the parameters with large numerical ranges from having an effect on the learning technique; all features thus contribute equally. Min-Max normalization is, of course, standard processing — normally transforms into a common fixed range the numerical variables for the model to learn appropriately. Automation introduces normalization process above, especially important to the peculiar task of vulnerability identification across smart contracts in Ethereum, since there is a big difference in numerical ranges of features.

This process limits with significant numerically ranged features. Ensure that all of them behave like this for the parameterization of the model. Denominators of normalization thus, end up transforming part of the features directly to the hexadecimal and binary representations of the bytecode at models using bytecode as input directly. These feature vectors are given to the detection phase for vulnerability detection. Generative normalization has been widely used nowadays as preprocessing techniques usually convert the larger numerical features into the common range by which the model should learn well. It is used in identifying vulnerabilities in smart Ethereum contracts because there is a very large difference in the range of numbers that different features can exhibit. With normalization, that will ensure that there is no bias from any particular feature, but rather equal contribution from all. Additionally, normalization improves the overall convergence speed of the training process, facilitating better performance in detecting vulnerabilities within smart contracts.

3.4. Vulnerabilities detection

The objective of this endeavor is to detect multiple vulnerabilities that potentially exist in a smart contract. To achieve this, system aim to utilize a diverse range of models, reducing the number of required training sessions while simultaneously identifying numerous vulnerabilities. By incorporating expanded features, the system uncovers vulnerabilities in previously unexplored patterns, in addition to detecting vulnerabilities in known patterns using current features. This enhanced approach improves the network scalability and significantly reduces training time. The proposed system encompasses 6 types of vulnerabilities.

3.4.1. Graph construction FeGNN

During graph construction, the nodes are differentiated into major nodes, fallback nodes, and secondary nodes. Major nodes characterize invocations to either customize or built-in functions that play a crucial role in detecting the specific vulnerability. For instance, a main node mimics the invocation of a transfer function in the case of the reentrancy vulnerability because these are crucial for identifying reentrancy. Similarly, in the context of endless loops, every tailored function within the contract is regarded as major nodes M . The critical functions as major nodes $M_1, M_2, \dots, M_n$ and secondary nodes are utilized to model crucial variables, such as user balance and bonus flag, whereas large nodes represent significant invocations. The critical parameters are formal defined as secondary nodes $S_1, S_2, \dots, S_n$. A fallback node F is added to model the behavior of the fallback function in an attack contract, allowing interactions with the function under test [23]. The fallback function is a special characteristic in SCs that frequently results in several security flaws. Then generate edges to represent the connections between nodes. The secular number of an edge indicates its position inside the contract function under test, and each edge provides a possible path that the function follow.

3.4.2. Contract graph normalization

Many GNNs inherently treat all nodes equally during information propagation, disregarding the reality that specific nodes have more central roles surpassing others. Additionally, the variability in contract source code results in different graph structures poses challenges for training GNN. To tackle these issues, a vertex exclusion process aimed at normalizing graphs (see Fig. 3).

A graph's node is divided into major $\{M_i\}_{i=1}^{|M|}$, secondary $\{S_i\}_{i=1}^{|M|}$, and fallback node F . However, the feature is transferred to the closest main node M_i upon the removal of each secondary node S_i . The feature is transmitted to all the closest significant nodes

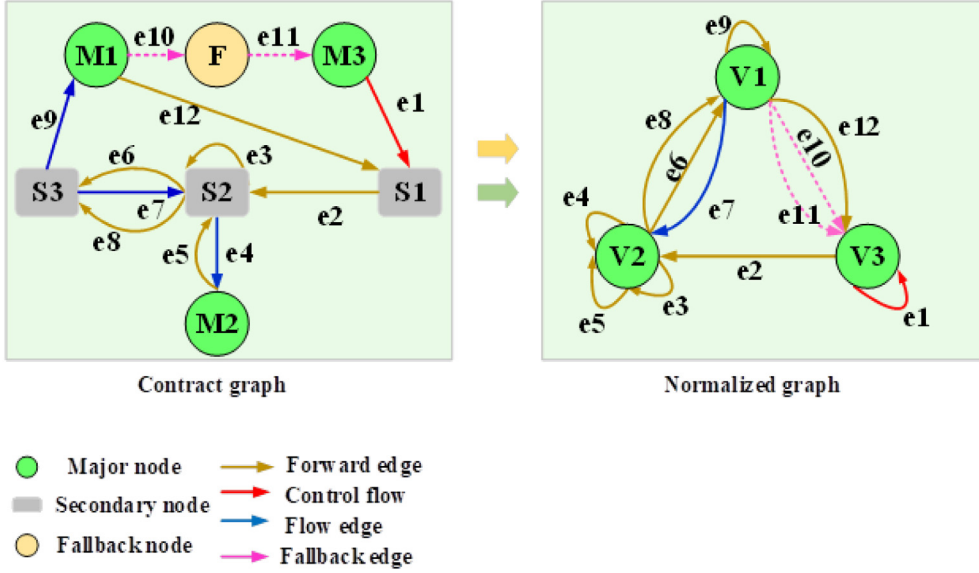


Fig. 3. Graph generation process and the resulting normalized graph for vulnerability and code clone detection.

Semantic fact	Type
assert { X }	Control-flow edges
require { X }	
revert	
throw	
if { X }	
if{...} else { X }	
if{...} then { X }	
while { X } do{...}	
for { X } do{...}	
assign { X }	Data-flow edges
access { X }	
natural sequential relationships	Forward edge
interactions with fallback function	Fallback edge

if multiple options are present nearest major nodes. The fallback node is eliminated in a manner similar to that of the secondary node. When a node is eliminated, its edges are still connected to other nodes, but their starting or ending nodes are shifted to the matching major node are shown in Fig. 2.

The features of major nodes are updated through the aggregation of features from its nearby removed nodes. The new major node of M_i as V_i is denoted as relation between the original major node and its related node upon aggregation. The characteristics of V_i is classified into (i) self-feature, (ii) in feature — features of the secondary nodes $\{S_i\}_{i=1}^{|M|}$ that merged into M_i and have a path leading from k_j to M_i ; and (iii) out-feature-characteristics of the secondary nodes $\{R_k\}_{k=1}^{|R|}$ that are merged with M_i and have a direct path from R_k to M_i [36].

The specifics of the semantic edges are provided in Table 3. Edges are categorized into four types: control-flow, data-flow, forward, and fallback. The constructed graph is passed to vulnerability detection.

3.4.3. Vulnerability detection phase

In the method presented, a scenario involving a distributed graph configuration is addressed. This involves either partitioning a single graph or distributing multiple graphs across numerous edge servers. These edge servers cannot be concentrated for training purposes, primarily due to security or regulatory limitations. The introduced model is based on GNNs and expands upon prior research involving Federated Learning (FL) implemented with neural network system in different territory. Therefore, the proposed FeGNN have two distinct stages: a readout stage and a message-passing stage [37]. In Fig. 4 the formulation of FeGNN in separate phases are displayed.

Phase 1, the information-passing phase, comprise of two essential stages: (i) the system collects and transfer messages from neighboring nodes, and (ii) the model employs the aggregated messages to update the hidden states of the nodes. Each edge is

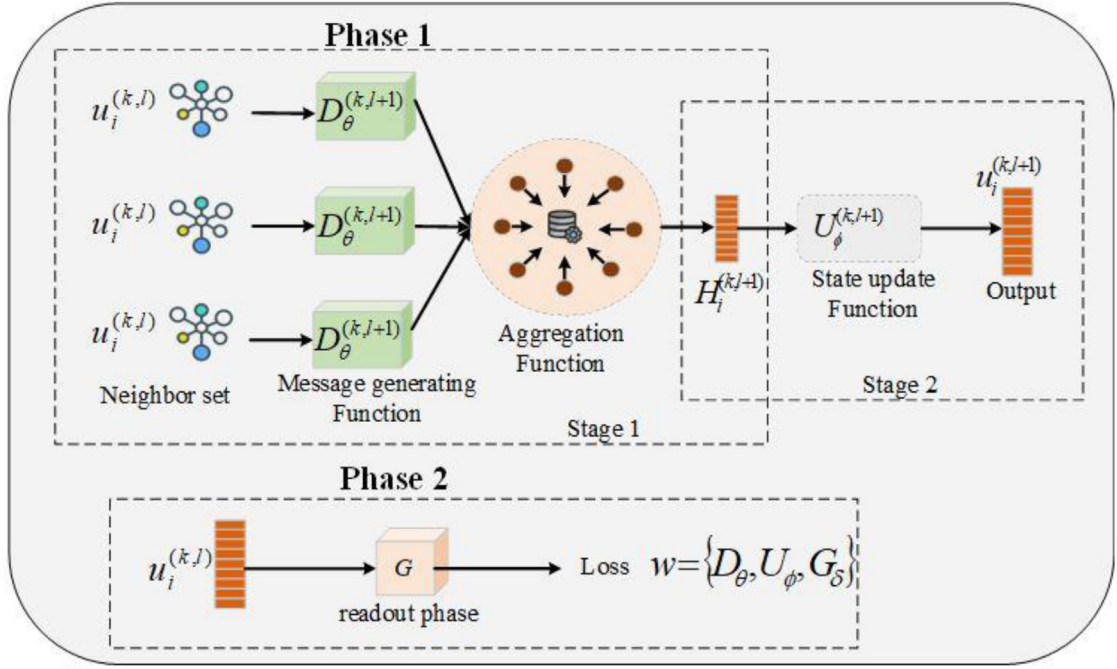


Fig. 4. Formulation of Federated Graph Neural Networks.

processed in a single time step when messages are passed along them. The layer indices are formalized by Eqs. (4),(5).

$$H_i^{k,l+1} = A_g \left(\left\{ D_\theta^{k,l+1} \left(u_i^{(k,l)}, u_j^{(k,l)}, b_{i,j} \right) \mid j \in N_i \right\} \right) \quad (4)$$

$$u_i^{(k,l+1)} = U_\phi^{(k,l+1)} \left(u_i^{(k,l)}, H_i^{(k,l+1)} \right) \quad (5)$$

where, layer index is denoted as l , aggregation function is represented as A_g , N_i denotes the neighbor set of nodes i , and neighbor j node hidden state is represented as $u_j^{(k,l)}$. The message generating function $D_\theta^{(k,l+1)}(\cdot)$ uses the hidden state of the current node $u_i^{(k,l)}$ as its input and the edge features is denoted as $b_{(i,j)}$ and the state update function $U_\phi^{(k,l+1)}(\cdot)$ getting the aggregated feature $H_i^{(k,l+1)}$. $u_i^{(k,l+1)}$ is final output of phase 1.

In phase 2, once all edges traversed, the readout phase calculates a label for the feature vector based on the node's hidden states, u_i^T represents the i th node's final hidden state, the prediction label $\hat{y}$ is generated by Eq. (6),

$$\hat{y} = \arg \max_i \sum_{l=1}^{|P|} f(u_i^T) \quad (6)$$

where, mapping function is represented as f , and $|P|$ indicates the major nodes quantity. The classification formula, presented in Eq. (7), assigns varying weights to each classifier and predicts class labels based on the predicted probabilities. Each vulnerability is assigned an index: 0 for safe, 1 for access-control, 2 for arithmetic, 3 for other, 4 for reentrancy, and 5 for unchecked-calls. The vulnerability class to which the sample belongs is the weights with the maximum probability value.

On the other hand, the variations between the original concealed state u_i^0 and the final internal state u_i^T are instructive in the vulnerability identification work. To formulate the FL setting, the GNN learnable weights are defined as $w = \{D_\theta, U_\phi, G_\delta\}$. D_θ is discriminative function, U_ϕ is update function, and G_δ is gradient function. It is constructed in Eq. (7).

$$\min_w F(w) \quad \text{def} \quad \min_w \sum_{k=1}^K \frac{N^{(k)}}{N} f^{(k)}(w) \quad (7)$$

where, $f^{(k)}(w) = \frac{1}{N^{(k)}} \sum_{i=1}^{N^{(k)}} \Gamma(w; V_i, k_j, R_k)$ that measures the clone and vulnerability detection over the graph dataset with data samples ($N^{(k)}$). $F(w)$ is objective function. $\text{def} \min_w$ is define $F(w)$ as the minimum value of $F(w)$ with respect to w . N is the total number of data samples across all subsets. K is the number of subsets or groups in the dataset. The global GNN model loss function is represented as Γ . Table 4 shows the pseudo code for vulnerability detection using FeGNN.

FeGNN processes smart contracts by representing them as graphs, where nodes symbolize code elements like functions and variables, and edges depict relationships such as data dependencies and control flow paths. It aggregates information from neighboring nodes to learn contextual embeddings, representing the code's semantics. FeGNN captures both data flow semantics

Table 4

Pseudo code for vulnerability detection using FeGNN.

```

Pseudo code for vulnerability detection using FeGNN


---


Initialize class AAST node, class GraphNode, class GraphEdge, and class FeGNN
Initialize FeGNN model parameters
    constructor ():
        function forward (graphNodes, graphEdges)
def constructAAST
def constructGraph(AAST)
def clientUpdate(client_data, fegnn)
graph_list = preprocessData(client_data)
    for graph in graph_list:
graphNodes, graphEdges = preprocess_graph(graph)
    predictions = fegnn.forward(graphNodes, graphEdges)
    // Update FeGNN model using client's local data
def federatedAveraging(models)
    // Aggregate model updates using federated averaging def main ()
    // Load and preprocess data
global_data = load_global_data ()
fegnn = FeGNN ()
    // Create a list of client datasets
    clients = split_global_data_into_clients ()
    for round in range(num_rounds):
client_models = []
    for client in clients
client_update (client. Data, fegnn. clone ())
client_models. append (client. updated model)
    // Aggregate client updates
aggregated_model = federatedAveraging(client_models)
    // Update global model with aggregated model
fegnn. update_with(aggregated_model)
    end
end
    // Detect vulnerabilities using the global model
    for local_data in distributed_data:
graph_list = [constructGraph(constructAAST(code)) for code in local_data]
    for i in range(len(local_data)):
        graph = graph_list[i]
vulnerability_score = global_model.predict (graph)
    if vulnerability_score threshold:
        print ("Vulnerability detected:", local_data[i])
    end
end
end

```

tracking the movement and transformation of data within the contract and control flow semantics, which involves understanding execution order and control structures. The model normalizes contract graphs for consistency across different contracts, enhancing its effectiveness in tasks like vulnerability detection and code clone identification.

For contract vulnerability detection training, BAST-FeGNN is used. The networks are given a large number of normalized graphs created from smart contract functions during the training phase, together with the accompanying ground truth labels. Once the models are trained, models are engaged to process a normalized graph and provide vulnerability detection labels as well as clone code detection. By obtaining the last graph-level characteristics angle for a purpose, vulnerabilities are detected. Fig. 5 shows the BAST-FeGNN architecture for vulnerability detection.

The core idea of this proposition is a combination of BAST and FeGNN and the creation of a new approach involving hybridization of this model. It is aimed at safeguarding Ethereum SCs via the detection of multiclass vulnerabilities and code clones. Combining AAST and FeGNN enables the construction of a robust and efficient model, BAST-FeGNN. AASTs are designed for semantic analysis of SCs. AASTs, thus enable an in-depth analysis of the structure and behavior of SCs and thus facilitate the study of their behavior. For graph normalization and feature extraction, they are excellent-the resulting processing of complex contract data is guaranteed to be precise and efficient.

Indeed, this dual use becomes even more important with this hybridization, as it would truly combine the very best of both worlds. The first of which would be that of capturing even the most nuanced semantic detail by AASTs that is very crucial to detecting subtle vulnerabilities. The other benefit comes in the form of an excellent mechanism by FeGNN to process massive amounts of data and extract features with high quality. Normalization by min-max integrates normalized features for training, thus optimizing its procedure, while the application of FL enables decentralized model training, preserving data privacy and improving scalability. With this combination, the hybrid BAST-FeGNN offers an all-around strategy for identifying various kinds of vulnerabilities and code clones in Ethereum SCs. This leads to improvement not just in precision and coverage but in robustness and reliability of blockchain-based applications, securing much more efficiently and dependably their operation.

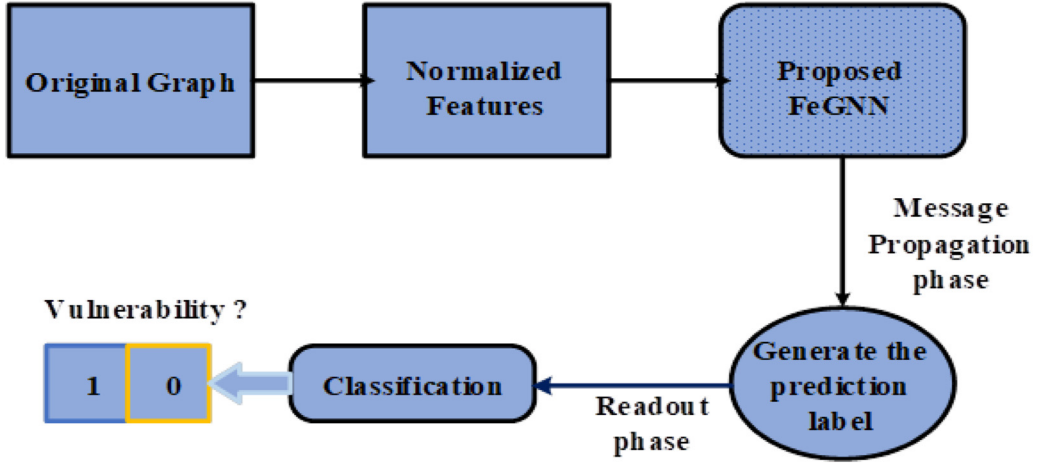


Fig. 5. BAST-FeGNN architecture for vulnerability detection, integrating feature extraction and graph neural networks.

Table 5

Description of factor.

Variable	Tuning values	Setting
Layer type	–	Conv2D, MaxPooling2D, BatchNorm2D
Total trainable parameters	–	2,0243,042
Loss function	–	Binary cross entropy
Learning rate	[0.01, 0.001, 0.0001]	0.001
Optimizer	–	Stochastic gradient descends
Epochs	[100, 200, 300, 400]	200
Batch size	[8, 16, 32, 48]	16
Dropout	[0.1, 0.2, 0.3, 0.4]	0.1
L2 regularization	$[\lambda^{-1}, \lambda^{-2}, \lambda^{-3}, \lambda^{-4}]$	λ^{-4}
Embedding size	[4, 8, 16, 32]	16

4. Result and discussion

4.1. Dataset description

Since the emergence of SC vulnerability differentiation is a relatively recent development, there exists a scarcity of open-base datasets containing labeled SCs; furthermore, the available datasets tend to be of modest proportions. Therefore, the slither audited smart contracts dataset [38], SmartBugs dataset [21], Solidity Fault Injection (SolidiFI) dataset [15] is used in this system. Slither-audited-smart-contracts is built to provide a freely available large-scale dataset for vulnerability detection and classification on verified solidity smart contracts. The base code for each contract is obtained either from the mentioned depository or through the Etherscan API². Meanwhile, the bytecode is acquired utilize the web3 python library, enabling interaction with a remote Ethereum or local node. More than 100,000 Ethereum chain validated SCs are acquired and labeled throughout the dataset development phase. To ensure balanced representation, the dataset is partitioned into validation, training, and test sets while maintaining the class proportions. As a result, proposed dataset consists of 79.6k, 10.8k, and 15.9k elements in the validation, training and test sets, respectively. This distribution maintains class proportions across sets to ensure that the learning process is both balanced and reflective of the diversity of contract types and vulnerabilities present in the dataset. This careful partitioning allows for robust model training and evaluation, ensuring that proposed detection techniques are well-tested and generalizable across various smart contract scenarios.

4.2. Experimental setup

All the investigations are organized on a computer provide with an Intel(R) Core (TM) i7-6900K, 8/16 cores/threads, and 64 GB DDR4 of Memory. The proposed BAST-FeGNN method for identifying vulnerabilities in SCs includes the construction of graph representations and the extraction of patterns.

The overall smart contract vulnerability experiment is conducted on python tool. The smart contract with BAST-FeGNN model is experimented with pytorch based on the digital library. Table 5 gives the description of the factors.

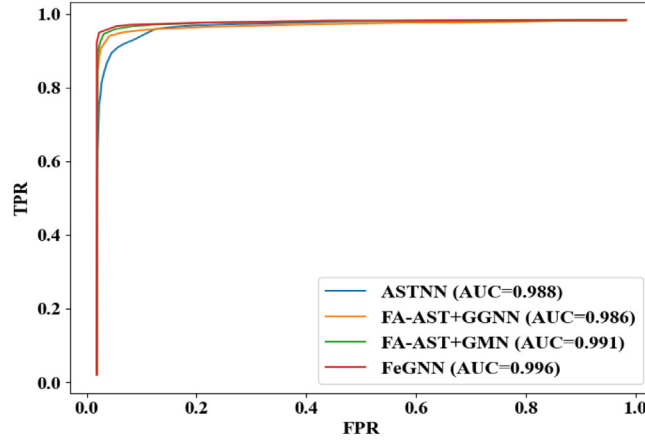


Fig. 6. ROC curve analysis for evaluating the performance of the proposed vulnerability detection model.

4.3. Evaluation metrics

Accuracy, precision, F1-score and recall matrices are utilized to measure the execution of the BAST-FeGNN model. True Negatives (TN), True Positives (TP), False Negatives (FN), and False Positives (FP) results serve as the baseline values for other measures. The number of accurately detected outcomes are true negative or true positive represents the true values. The incorrect outputs from the DL model are indicated by the false values.

> Precision

The ratio of actually positive values to all positive detection is described by the precision P metric as shown in Eq. (8). This demonstrates the accuracy of the classification algorithm's optimistic prediction. The section of accurately predicted vulnerable trial to those predicted to be vulnerable.

$$P = \frac{TP}{TP + FP} \quad (8)$$

> Recall

The percentage of accurately predicted vulnerable samples to all vulnerable samples. The formula for finding the recall R is shown in Eq. (9)

$$R = \frac{TP}{TP + FN} \quad (9)$$

> F1-score

It assesses the comprehensive effect by taking both recall and precision into account. Widely utilized in information retrieval, this metric gauge's overall decision accuracy through a combination of recall and precision. The F1-score is evaluated as the harmonic mean of recall and precision which is expressed in Eq. (10).

$$F1 - score = \frac{2 \times P \times R}{P + R} \quad (10)$$

The F1-score ranges from 1 to 0, with 0 being the best value. Each class label's F1-score is determined individually or all at once.

> Accuracy

The proportion of all samples that are detected correctly. The formula for finding the accuracy Acc is shown in Eq. (11)

$$Acc = \frac{TP + TN}{TP + TN + FP + FN} \quad (11)$$

4.4. Performance evaluation

By balancing the relative costs of the False Positive Rate (FPR) and True Positive Rate (TPR), where $TPR = TP/(TP + FN)$ and $FPR = FP/(TN + FP)$, Receiver Operating Characteristic (ROC) curves are used to evaluate the execution of contract ward.

Fig. 6 illustrates ROC curves using TPR as the y-axis and FPR as the x-axis. The optimum point, according to theory between (0, 1), which signifies the positive and negative samples are correctly identified, respectively. As a result, the result of detected vulnerabilities is better, as indicated by the point closer to top left. The proposed FeGNN model achieves 0.996 which is better than the existing FA-AST+GMN [37], FA-AST+GGNN [36] and ASTNN [36].

The BAST-FeGNN model's superior performance, indicated by the high ROC score of 0.996, results from its enhanced ability to capture structural and semantic dependencies within smart contract code. This approach allows it to precisely distinguish

Table 6
Comparative analysis of clone detection results.

Model	Precision (%) $\uparrow$	Recall (%) $\uparrow$	F1-score (%) $\uparrow$
ASTNN [36]	92	94	93
CDLH [37]	92	74	82
AST+GGNN [36]	85	90	88
AST+GMN [37]	96	94	95
Proposed	98.44	97.19	97.81

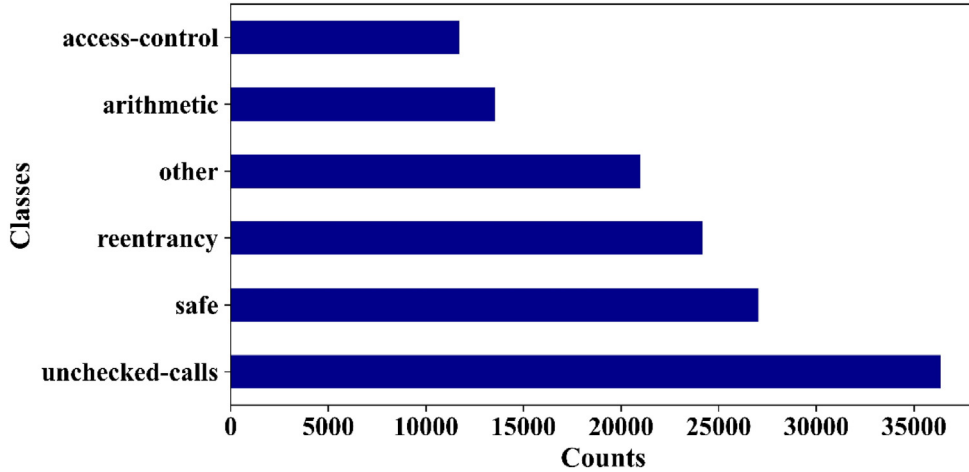


Fig. 7. Class distribution of different vulnerabilities, showing the frequency of each vulnerability type.

vulnerabilities by effectively leveraging feature-based graph neural networks, outperforming existing models in identifying nuanced patterns within complex code structures.

Table 6 describes the comparative analysis of clone detection results for different models in terms of accuracy, precision, F1-score and recall. The proposed model appears to outperform the other models in expression of precision (98.44%), recall (97.19%), and F1-score (97.81%). It achieves high values in these metrics, indicating that it is capable of accurately identifying true clones while also minimizing false negatives. This suggests that the introduced model is more important at clone prediction compared to the other models such as ASTNN [36], CDLH [36], AST+GGNN [36], and AST+GMN [37]. The proposed BAST-FeGNN achieves 98.10% accuracy for the clone detection task.

To better compare the detection performance of vulnerabilities, the class distribution results of the various vulnerabilities are displayed in Fig. 7. The figure compares the vulnerabilities such as access control, arithmetic, other, reentrancy, safe, and unchecked calls. The findings indicate that the unchecked calls are the highest detection rates of 35000, the access control arithmetic demonstrates relatively lower detection rates between 10000 and 15000. Safe vulnerabilities detection rates between 25000 and 30000. Reentrancy and other vulnerabilities have the detection rate between 20000 and 25000.

In Fig. 8 illustrates the accuracy and loss towards epoch in the training and validation state. Moreover, the train and validating value of accuracy is suddenly peaked at the value of 1.0 and make constant of epoch rate 200. The loss of training and validation state ranges from 0.1 to 0.7 gradually towards the 200-epoch rate.

The training loss curve indicates the evolution of the loss function over each iteration or epoch of the training process. The loss function calculates the difference among the model's detected output and the actual target labels. The objective of training is to minimize this loss function, effectively making the model more accurate. Initially, the training loss is relatively high as the model makes random predictions. This improvement arises from BAST-FeGNN's graph-based feature extraction, which enhances accuracy by effectively capturing and representing vulnerabilities, surpassing traditional sentiment analysis models in performance.

Fig. 9 illustrates the confusion matrix for vulnerabilities like access-control, arithmetic, other, safe and unchecked-calls. This confusion matrix is to predict the accuracy score. The information points are expressed visually throughout many categories to demonstrate correlation in terms of correlations. Multiple sets of feature correlation coefficients are shown in a correlation matrix. To demonstrate the association among multiple characteristics and various data points, five groups are displayed. It is possible to choose an algorithm for categorization based on the structure of the information by measuring the correlation among the data [39,40]. The training of data system using linear regression, for instance, does not yield satisfactory results if the data do not have a good linear correlation.

The comparison of false positive rate and true positive rate for introduced and existing technique are display in Fig. 10. The false positive rate and true positive rate are represented in y-axis and x-axis respectively. The BAST-FeGNN system performs better than other systems like optimized CNN, optimized LSTM and optimized-CodeBERT model. Using the slither-audited smart contract

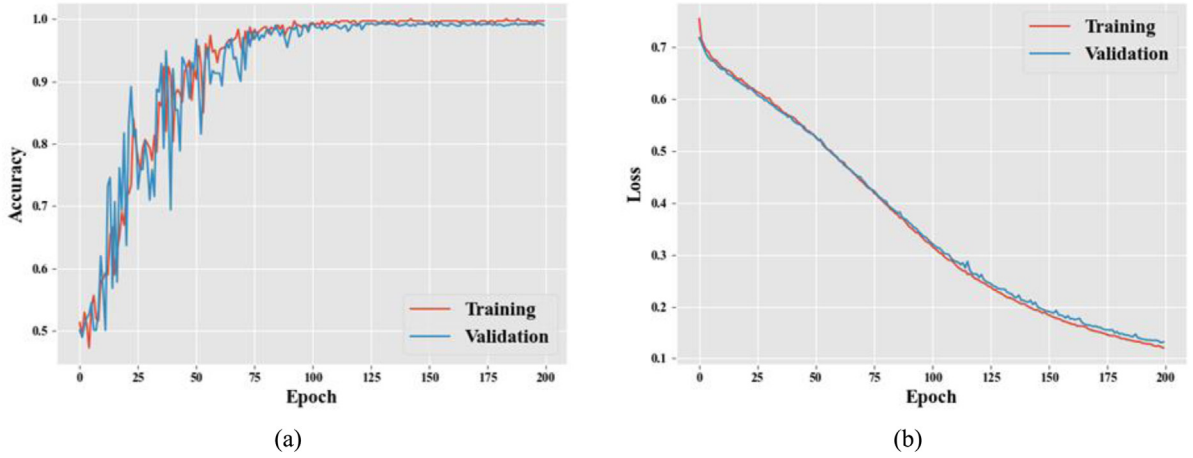


Fig. 8. Accuracy and loss trends over epochs during training and validation processes.

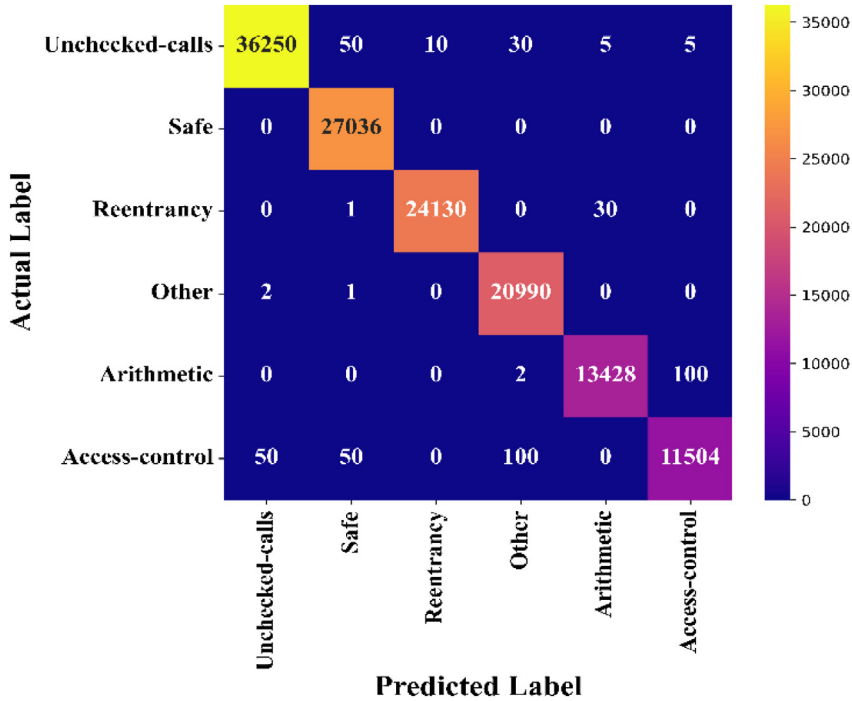


Fig. 9. Confusion matrix for vulnerabilities, illustrating the model's classification performance across different vulnerability classes.

dataset, the proposed BAST-FeGNN model's execution increases over other systems, as shown in the figure. The BAST-FeGNN model outperforms other models due to its advanced feature extraction capabilities, which allow it to minimize false positives and enhance true positive rates. By capturing intricate code patterns, it provides superior detection accuracy compared to models.

The above Fig. 11 shows that the execution of error rate among the introduced and existing technique. The error rate for the optimized-LSTM has the higher error rate than the other methods. In the proposed BAST-FeGNN technique has lower error rate than the other optimized-CNN, LSTM and CodeBERT of existing approaches. The BAST-FeGNN achieved the best performance with 0.02 error rate than the other approaches.

The BAST-FeGNN model achieves a 0.02 error rate by leveraging an advanced graph-based model, which efficiently captures semantic and structural features within smart contract code. This approach enhances model precision and robustness, reducing misclassifications compared to optimized CNN, LSTM, and CodeBERT, thus demonstrating superior detection capability on vulnerability-specific tasks (see Fig. 12).

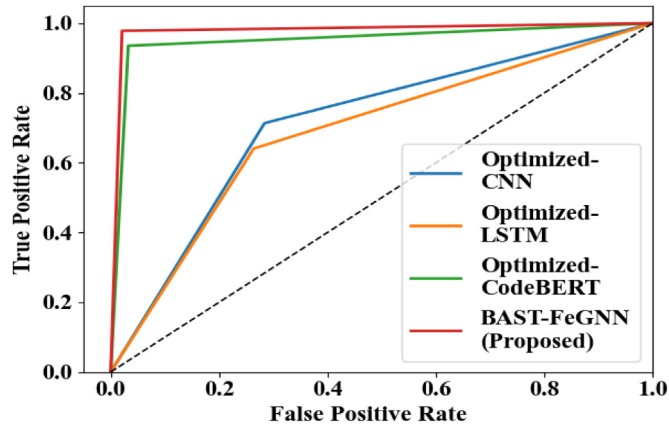


Fig. 10. Comparison of true and false positive rates between the introduced and existing techniques.

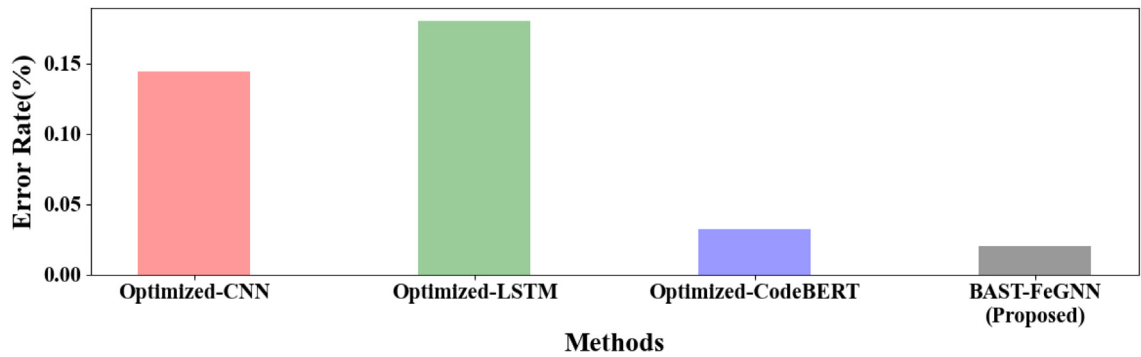


Fig. 11. Performance comparison of error rates between the proposed and existing techniques.

Table 7

Existing works comparative analysis on slither audited smart contracts dataset.

Methods	Accuracy (%) ↑	Precision (%) ↑	F1-score (%) ↑	Recall (%) ↑
Optimized-CNN [26]	85.54	71.61	70.62	71.36
Optimized-LSTM [26]	81.96	73.61	63.05	64.06
Optimized-CodeBERT [26]	96.77	96.77	93.53	93.55
SCVHunter [25]	90.65	87.52	84.13	85.79
BAST-FeGNN (Proposed)	98.0	97.91	95.74	97.55

In Fig. 11 shows that the performance of accuracy, precision, F1-score, recall for the proposed and existing technique. In Fig. 11(a) describes the accuracy level of optimized-CNN [26] has 85.54%, optimized-LSTM has 81.96%, optimized-CodeBERT [26] has 96.77%, BAST-FeGNN [26] has 98.0%, and SCVHunter [25] has 90.65%. And in Fig. 11(b) describes the precision level of optimized-CNN has 71.61%, optimized-LSTM has 73.61%, optimized-CodeBERT has 96.77%, BAST-FeGNN has 97.91%, and SCVHunter has 87.52%. In Fig. 11(c) describes the F1-score level of optimized-CNN has 70.62%, optimized-LSTM has 63.05%, optimized-CodeBERT has 93.53%, BAST-FeGNN has 95.74%, and SCVHunter has 84.13%.

As well as in Fig. 11(d) describes the recall level of optimized-CNN has 71.36%, optimized-LSTM has 64.06%, optimized-CodeBERT has 93.55% and BAST-FeGNN has 97.55%, and SCVHunter has 85.79%. As compared to the other technique the proposed BAST-FeGNN technique has high accuracy rate, precision rate, F1-score rate and recall rate.

The BAST-FeGNN model achieves superior performance metrics accuracy (98.0%), precision (97.92%), F1-score (95.74%), and recall (97.55%) by employing advanced graph-based feature aggregation and node representation learning techniques. These mechanisms enable BAST-FeGNN to extract high-dimensional, context-aware embeddings, thereby improving discriminative capacity for nuanced vulnerability patterns over models like CNN, LSTM, and CodeBERT.

Table 7 describe the introduced model demonstrates outstanding accuracy, precision, F1-score, and recall as 98.0%, 97.91%, 95.74% and 97.55%, respectively.

Table 8 presents the performance metrics on the SmartBugs dataset, highlighting proposed method's superior accuracy (2.91%), precision (3.71%), F1-score (8.56%), and recall (6.32%) over existing methods. Table 9 presents the results obtained on the Solidify

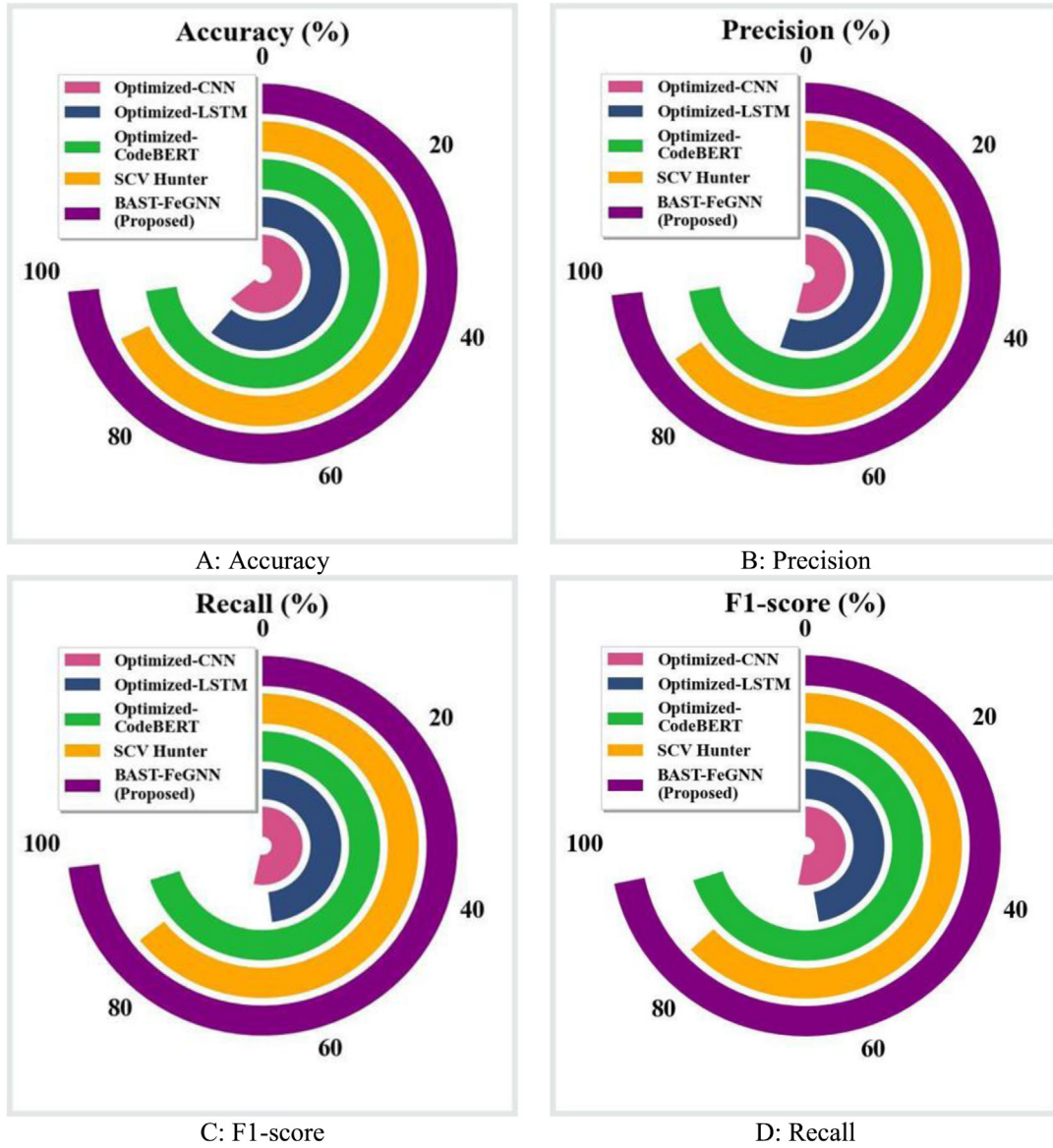


Fig. 12. Performance analysis of (a)Accuracy, (b)Precision, (c)F1-score and (d)Recall for the proposed system and existing models.

Table 8
Existing works comparative analysis on SmartBugs dataset.

Methods	Accuracy (%) ↑	Precision (%) ↑	F1-score (%) ↑	Recall (%) ↑
SmartGuard [21]	91.40	92.24	90.4	91.31
Bi-GRU [22]	96.1	95.3	97.8	96.5
BERT [24]	98.7	98.7	86.2	90.9
GNN [30]	77.90	78.00	77.5	77.00
HGAT [32]	85.64	76.96	82.47	88.12
BAST-FeGNN [Proposed]	98.9	98.91	98.14	97.12

dataset, demonstrating that the proposed approach significantly outperforms several existing methods, including CodeNet [15], ASG [17], MODNN [20], MLP+LSTM [27], and QuadraCode AI [28]. The proposed method achieves superior accuracy improvements of 4.82%, 33.30%, 4.10%, 57.64%, and 20.60%, respectively, compared to these baseline models. Moreover, it exhibits enhanced precision gains of 9.41%, 28%, 7.76%, 65.87%, and 22.11%, respectively. The F1-score is also elevated, with increases of 2.93%,

Table 9

Existing works comparative analysis on Solidify dataset.

Methods	Accuracy (%) ↑	Precision (%) ↑	Recall (%) ↑	F1-score (%) ↑
Code Net [15]	94.35	90.40	97.66	94.35
ASG [17]	74.19	77.27	89.47	82.93
MODNN [20]	95.0	91.78	95.12	93.42
MLP+LSTM [27]	41.89	59.63	43.37	40.17
QuadraCode AI [28]	82.00	81.00	78.00	75.00
BAST-FeGNN [Proposed]	98.9	98.91	98.14	97.12

Table 10

Methods and their computational complexity.

Method	Computational complexity ↓
CodeNet [15]	$O(N \times D)$
Bi-GRU [22]	$O(N \times D^2 \times k)$
Optimized-CodeBERT [26]	$O(N \times D \times k)$
BAST-FeGNN [Proposed]	$O(N)$

Table 11

Methods and their average execution time on SmartBugs dataset.

Algorithm	Average execution time (s) ↓		
	Reentrancy	Tx. Origin	Timestamp dependency
CodeNet [15]	0.13	0.13	0.15
ASG [17]	3.8	2.6	3.4
BAST-EeGND [Proposed]	0.11	0.11	0.13

17.11%, 3.80%, 58.63%, and 29.49%, respectively. Additionally, the recall rates show improvements of 0.49%, 9.69%, 3.07%, 55.80%, and 25.82%, respectively, surpassing those of the existing models evaluated.

It indicates that the method introduced is highly accurate in vulnerability detection in smart Ethereum contracts the overall observation shows very good performance of the method with respect to all metrics as existing systems lose out in the expression of the accuracy, precision, F1 score, or recall by the proposed method.

The presented BAST-FeGNN model has demonstrated a promising outcome due to its specific architecture utilizing a graph neural network to extract learned features and model relationships among smart contract codes. This, coupled with a multi-level attention incorporated into the model, helps elicit complex dependencies, which promotes better comprehension of vulnerabilities. The rigorous training regimen, as seen in the training and validation loss curves, yields optimal parameter tuning. Advancing techniques, when paired with an extensive dataset, make a solid hold for good generalization within the model. Thus, high accuracy, precision, recall, and F-measure values across a variety of datasets will prove leading against previous methods for vulnerability detection.

4.5. Computational complexity analysis

Computational complexity refers to the level of computational resources required to vulnerability and clone detection in Ethereum SCs. Comprehending computational complexity is vital when choosing algorithms that achieve a symmetrical adjustment between accuracy and efficiency.

Table 10 illustrates the computational complexity analysis of various methods, highlighting their respective computational complexities. Where, k represent the number of times the entire population goes through the update process, D denotes the dimensionality and n is the size. The assessment is conducted and contrasted with established methods. Notably, the BAST-FeGNN proposed approach exhibits lower complexity compared to existing methods.

Table 11 provides a comparison of the average execution times for three different algorithms on SmartBugs dataset used to detect three types of vulnerabilities in SCs: reentrancy, Tx. Origin, and timestamp dependency. The algorithms compared are CodeNet [15], ASG [17], and the proposed BAST-FeGNN. The results underscore the effectiveness of the BAST-FeGNN in optimizing the execution time while maintaining robust vulnerability detection capabilities.

Space complexity analysis among various algorithms has been provided in Table 12. The proposed BAST-FeGNN architecture consumed only 1.2 GB space, which is quite low as compared to existing architectures, thereby proving the efficiency of BAST-FeGNN in terms of memory consumption.

The architecture of BAST-FeGNN is really complex because it integrates BAST-FeGNN, which actually needs power resources for enormous computation. Although the binarization of attention mechanisms optimizes memory storage, it still needs high processing power, especially in primary training. Deploying the model on a blockchain network having various nodes with dissimilar computational capabilities certainly raises additional challenges. The mentioned problems have been solved by using several

Table 12
Space complexity analysis.

Algorithm	Memory Usage (GB) ↓
CNN [16]	2.5
BERT [24]	1.5
LSTM [26]	2.0
BAST-FeGNN [Proposed]	1.2

Table 13
Comparative performance of vulnerability detection models in DeFi context.

Model	Accuracy (%) ↑	Precision (%) ↑	Recall (%) ↑	F1-score (%) ↑	Unique Features
Optimized CodeBERT [26]	96.77	96.77	93.55	93.53	Code-targeted CNN pretrained on code snippets; limited for large-scale DeFi contracts.
ASG [17]	74.19	77.27	82.93	81.90	ASG-based bytecode analysis; not suited for high-volume transaction processing in DeFi.
SCVHunter [25]	90.65	87.52	85.79	84.13	GNN-based heterogeneous semantic graph; requires significant memory, high computational demands.
BAST-FeGNN [Proposed]	98.0	97.91	97.55	95.74	AST-based graph construction, FeGNN, efficient memory usage (1.2 GB). Delivers high accuracy and ideal for reentrancy and arithmetic vulnerability detection in DeFi and solves computational issue.

advanced techniques to minimize the computational expenditure at no cost to performance. The proposed work resolves the computational problem through stochastic gradient descent.

A critical factor per any solution that ought to be adopted on a blockchain network is scalability. The BAST-FeGNN model ought to seamlessly scale to handle the growing number of SCs and transactions. It is vital for such a model to work on several nodes while handling distributed data. Using federated learning in this context allows FeGNN to learn from data distributed on several nodes while avoiding centralization and thus maintaining data privacy and removing communication costs.

In summary, while the BAST-FeGNN model has requirements for inputs as a result of its intrinsic architecture, the combined efforts through advanced techniques and federated learning, however, empower scalability for its deployment in the dynamic and decentralized environment of blockchain networks.

4.6. Case study: Preventing exploitation in a decentralized finance platform

This case study is meant to show that BAST-FeGNN is, in fact, practical in the real world when it comes to applying it to a problem, like that of Ethereum smart contract security. A decentralized finance platform provides a suite of financial services, encompassing lending, borrowing, and trading, and all such services are offered and managed via smart contracts on the Ethereum blockchain. The platform has grown significantly, leading to an increase in both the number and complexity of SCs deployed.

The higher the user activity and assets locked inside smart contracts, the more it is becoming attractive to attack. A significant flaw in any of these contracts results in huge monetary loss, which damages the platform based on its image and people's trust in it.

The developers of the platform examine the contracts for vulnerabilities and clone detection via BAST-FeGNN before new smart contracts can be deployed on the platform. By means of AST-based analysis, BAST-FeGNN has identified important security patterns and structures a contract graph capturing the data flow as well control flow within the code. Table 13 shows the comparative performance and features of vulnerability detection models in Decentralized Finance (DeFi) context.

Outcome: BAST-FeGNN detects several instances of reentrancy vulnerabilities and unsafe arithmetic operations, which are overlooked by manual code reviews and other automated tools. Additionally, it identifies code clones that are inadvertently copied with slight modifications, leading to potential duplicated vulnerabilities. The development team addresses these issues before deployment, significantly reducing the risk of exploitation. Table 14 provides the comparison of vulnerability and Clone Detection using Smart contracts.

4.7. Wilcoxon signed-rank test

The Wilcoxon signed-rank test is a non-parametric statistical method used to compare two related samples, assessing whether the population mean ranks differ. In proposed model this test is used to evaluate the classification performance of the FeGNN model by comparing accuracy metrics before and after applying the model.

Table 14
Performance comparison of proposed model.

Methods	F1-score (%)	Recall (%)	Precision (%)
HomoDec [41]	79	84	78
LSTM [41]	54	59	50
SmartConDetect [24]	90	-	-
GRU [41]	54	59	49
VulnSense [30]	77	77	78
PropaDT [41]	82	86	81
Proposed	95	95	96

Table 15
Gathered information (prior to conducting the Wilcoxon signed-rank analysis)

Fold	Accuracy before (%)	Accuracy after (%)	Absolute difference	Rank
1	85.36	96.25	10.89	5
2	88.34	97.24	8.90	3
3	89.75	98.12	8.37	2
4	87.25	97.61	10.36	4
5	90.67	98.00	7.33	1

Table 16
Wilcoxon signed-rank test result.

Metric	Value
Number of samples	100
Test statistic	15
p-value	0.044
Results	Statistically significant ($p < 0.05$)
Conclusion	FeGNN improves classification performance.

Table 17
Ablation study of proposed model.

Methods	Accuracy (%) ↑	Precision (%) ↑	Recall (%) ↑	F1-score (%) ↑
Individual GNN	89.45	85.25	82.61	86.02
Individual FL	78.56	78.74	77.12	78.98
BAST-FeGNN (Proposed)	98.0	97.91	95.74	97.55

Table 15 shows model accuracy before and after FeGNN, with rank sums for improvements totaling 15 for positive differences.

Table 16 presents the results of the Wilcoxon signed-rank test. The analysis indicates a pvalue of 0.044, suggesting a significant difference in classification accuracy when using the FeGNN model compared to not using it. This statistical validation reinforces the claim that the FeGNN technique enhances classification performance. Its graph-based representation captures complex code relationships, enabling better feature extraction and interaction. Additionally, the decentralized learning approach enhances model robustness and generalization, contributing to superior vulnerability detection.

4.8. Ablation study

In the conducted ablation study, an investigation is undertaken into the efficacy of different elements within the suggested model to ascertain their influence on performance metrics. A novel approach termed BAST-FeGNN is introduced, synergizing GNNs with FL to enhance vulnerability detection in Ethereum smart contracts. The integration of GNN facilitates effective representation of smart contracts as graphs, with nodes denote code elements (functions, variables) and edges illustrate their relationships (data dependencies and control flow).

Leveraging FL, the FeGNN model maintains data privacy and enables decentralized training across multiple edge servers, addressing regulatory and security constraints.

Table 17 presents a detailed comparison of the BAST-FeGNN model's performance, both with and without each of its individual components, highlighting the contributions of each module to the overall effectiveness of the algorithm.

4.9. Limitations

1. **Computational Overhead:** BAST-FeGNN's reliance on advanced graph construction and normalization techniques introduces significant computational demands, especially for large-scale or complex smart contracts, which may limit real-time applicability.

2. **Dependency on High-Quality Labeled Data:** The framework's performance heavily depends on the availability of extensive, high-quality labeled datasets, which are often scarce in real-world scenarios, potentially affecting its scalability and generalizability.

5. Conclusion

Ultimately, the model that has been developed addresses crucially the significant issue of dealing with timely and accurate detection of vulnerabilities found in smart contracts because it really serves as a key to poke holes in the said feasibility of these smart contracts in security. The proposed model addresses the issue by BAST-FeGNN model for code clone prediction and also for multiclass smart contract vulnerabilities in an Ethereum environment. Through graphical-based representations and BAST-FeGNN, the model incorporates complex control flow and data flow semantics towards high accuracy in detecting malicious and security-critical activities. Thus, this BAST-FeGNN approach certainly plays a huge role in strengthening the security of the SCs of Ethereum through significant and timely prediction of vulnerabilities. Moreover, these serve valuable resources to developers and auditors who seek to find potential security flaws within smart contracts before they are deployed, thus reducing risks and assuring reliability and trust for any blockchain-based applications. This further research and experiments are needed to confirm and even expand the capabilities of the introduced system to turn into a critical part of smart contract security. The introduced is evaluated with multiple existing approaches such as optimized-CNN, optimized-LSTM, optimized-CodeBERT, etc. The detection findings have proven efficacy and stability of the model for its applicability for identifying smart contract vulnerabilities. This intended research will improve the hypothesized neural model as well as look into other representations of programs. Merging AASTs with other program structures like data dependency graphs or token sequences can be another direction for current research.

5.1. Future scope

To enhance the effectiveness of vulnerability identification, future research should consider incorporating a broader range of graph representations. Future work will extend the methodology to smart contracts beyond Ethereum, enhancing its generalizability and applicability.

- **Multi-Platform Smart Contract Support:** The current approach is optimized for Ethereum smart contracts. A key area of improvement will be extending the BAST-FeGNN model to support smart contracts across other blockchain platforms such as Binance Smart Chain, Solana, and Hyperledger. This will enhance the system's versatility and make it applicable to a wider range of decentralized applications.
- **Real-Time Detection and Dynamic Analysis:** Currently, BAST-FeGNN is designed for static analysis before deployment. Expanding the system to include real-time or dynamic analysis capabilities will enable continuous monitoring of smart contracts during execution, detecting vulnerabilities that only manifest in runtime conditions. This will provide a proactive layer of security for decentralized applications.
- **Scalability Enhancements for Large-Scale Contracts:** While BAST-FeGNN addresses scalability to some extent, additional optimizations such as distributed computing models or GPU-accelerated training will further reduce computation time and make the system more suitable for analyzing extremely large smart contracts in real-world applications.
- **Optimization for Efficiency:** Future work could focus on optimizing the graph construction and feature extraction processes to reduce computational overhead, enabling real-time vulnerability detection for large-scale smart contracts.
- **Development of Semi-Supervised Learning Models:** To address the dependency on labeled datasets, exploring semi-supervised or unsupervised learning techniques could allow the framework to generalize better with limited labeled data, broadening its practical applicability.

CRedit authorship contribution statement

Shruti Sharma: Conceptualization, Methodology, Validation. **Ankur Ratmele:** Project administration, Supervision, Investigation. **Abhay Deep Seth:** Writing – original draft, Writing – review & editing.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare they have no conflicts of interest. All authors certify that they have no affiliations with or involvement in any organization or entity with any financial interest or non-financial interest in the subject matter or materials discussed in this manuscript.

Acknowledgment

None.

Data availability

The data supporting the findings of this study are cited and the link is listed in the references.

References

- [1] Srinivasan P. TP-detect: trigram-pixel based vulnerability detection for ethereum smart contracts. *Multimedia Tools Appl* 2023;1–15. <http://dx.doi.org/10.1007/s11042-023-15042-4>.
- [2] Narayana KL, Sathiyamurthy K. Automation and smart materials in detecting smart contracts vulnerabilities in blockchain using deep learning. *Mater Today: Proc* 2023;81:653–9. <http://dx.doi.org/10.1016/j.matpr.2021.04.125>.
- [3] Liu L, Tsai W-T, MdZ Bhuiyan, Peng H, Liu M. Blockchain-enabled fraud discovery through abnormal smart contract detection on ethereum. *Futur Gener Comput Syst* 2022;128:158–66. <http://dx.doi.org/10.1016/j.future.2021.08.023>.
- [4] Zhang L, Wang J, Wang W, Jin Z, Zhao C, Cai Z, et al. A novel smart contract vulnerability detection method based on information graph and ensemble learning. *Sensors* 2022;22(9):3581. <http://dx.doi.org/10.3390/s22093581>.
- [5] Zhang H, Zhang W, Feng Y, Liu Y. SVScanner: Detecting smart contract vulnerabilities via deep semantic extraction. *J Inf Secur Appl* 2023;75:103484. <http://dx.doi.org/10.1016/j.jisa.2023.103484>.
- [6] Guo R, Chen W, Zhang L, Wang G, Chen H. Smart contract vulnerability detection model based on siamese network (SCVSN): A case study of reentrancy vulnerability. *Energies* 2022;15(24):9642. <http://dx.doi.org/10.3390/en15249642>.
- [7] He D, Wu R, Li X, Chan S, Guizani M. Detection of vulnerabilities of blockchain smart contracts. *IEEE Internet Things J* 2023;10(14):12178–85. <http://dx.doi.org/10.1109/jiot.2023.3241544>.
- [8] Tian Z, Tian J, Wang Z, Chen Y, Xia H, Chen L. Landscape estimation of solidity version usage on ethereum via version identification. *Int J Intell Syst* 2022;37(1):450–77. <http://dx.doi.org/10.1002/int.22633>.
- [9] Zhang L, Li Y, Jin T, Wang W, Jin Z, Zhao C, et al. SPCBIG-EC: A robust serial hybrid model for smart contract vulnerability detection. *Sensors* 2022;22(12):4621. <http://dx.doi.org/10.3390/s22124621>.
- [10] Ince P, Luo X, Yu J, Liu JK, Du X. Detect llama-finding vulnerabilities in smart contracts using large language models. In: *InAustralasian conference on information security and privacy*. 2024, p. 424–43. http://dx.doi.org/10.1007/978-981-97-5101-3_23.
- [11] Chen D, Feng L, Fan Y, Shang S, Wei Z. Smart contract vulnerability detection based on semantic graph and residual graph convolutional networks with edge attention. *J Syst Softw* 2023;202:111705. <http://dx.doi.org/10.1016/j.jss.2023.111705>.
- [12] Zhu D, Yue F, Pang J, Zhou X, Han W, Liu F. Bytecode similarity detection of smart contract across optimization options and compiler versions based on triplet network. *Electron* 2022;11(4):597. <http://dx.doi.org/10.3390/electronics11040597>.
- [13] Hao Z, Zhang B, Mao D, Yen J, Zhao Z, Zuo M, et al. A novel method using LSTM-RNN to generate smart contracts code templates for improved usability. *Multimed Tools Appl* 2023. <http://dx.doi.org/10.1007/s11042-023-14592-x>.
- [14] Liu Z, Qian P, Wang X, Zhuang Y, Qiu L, Wang X. Combining graph neural networks with expert knowledge for smart contract vulnerability detection. *IEEE Trans Knowl Data Eng* 2021;1. <http://dx.doi.org/10.1109/tkde.2021.3095196>.
- [15] Hwang S-J, Choi S-H, Shin J, Choi Y-H. CodeNet: Code-targeted convolutional neural network architecture for smart contract vulnerability detection. *IEEE Access* 2022;10:32595–607. <http://dx.doi.org/10.1109/access.2022.3162065>.
- [16] Huang J, Zhou K, Xiong A, Li D. Smart contract vulnerability detection model based on multi-task learning. *Sensors* 2022;22(5):1829. <http://dx.doi.org/10.3390/s22051829>.
- [17] Zhang Y, Liu D. Toward vulnerability detection for ethereum smart contracts using graph-matching network. *Futur Internet* 2022;14(11):326. <http://dx.doi.org/10.3390/fi14110326>.
- [18] Qian S, Ning H, He Y, Chen M. Multi-label vulnerability detection of smart contracts based on bi-LSTM and attention mechanism. *Electron* 2022;11(19):3260. <http://dx.doi.org/10.3390/electronics11193260>.
- [19] Wu H, Dong H, He Y, Duan Q. Smart contract vulnerability detection based on hybrid attention mechanism model. *Appl Sci* 2023;13(2):770. <http://dx.doi.org/10.3390/app13020770>.
- [20] Zhang L, Wang J, Wang W, Jin Z, Su Y, Chen H. Smart contract vulnerability detection combined with multi-objective detection. *Comput Networks*. 2022;217:109289. <http://dx.doi.org/10.1016/j.comnet.2022.109289>.
- [21] Li Z, Li W, Li X, Zhang Y. StateGuard: Detecting state derailment defects in decentralized exchange smart contract. In: *Companion Proc the ACM Web Conf* 2024;2024:810–3.
- [22] Jain VK, Tripathi M. An integrated deep learning model for ethereum smart contract vulnerability detection. *Int J Inf Secur* 2024;23(1):557–75. <http://dx.doi.org/10.1007/s10207-023-00752-5>.
- [23] Zhuang Y, Liu Z, Qian P, Liu Q, Wang X, He Q. Smart contract vulnerability detection using graph neural networks. In: *Proceedings of the twenty-ninth international conference on international joint conferences on artificial intelligence*. 2021, p. 3283–90.
- [24] Jeon S, Lee G, Kim H, Woo SS. Design and evaluation of highly accurate smart contract code vulnerability detection framework. *Data Min Knowl Discov* 2023;38(3):888–912. <http://dx.doi.org/10.1007/s10618-023-00981-1>.
- [25] Luo F, Luo R, Chen T, Qiao A, He Z, Song S, et al. Scvhunter: Smart contract vulnerability detection based on heterogeneous graph attention network. In: *Proceedings of the IEEE/ACM 46th international conference on software engineering*. 2024, p. 1–13. <http://dx.doi.org/10.1145/3597503.363921>.
- [26] Tang X, Du Y, Lai A, Zhang Z, Shi L. Deep learning-based solution for smart contract vulnerabilities detection. *Sci Rep* 2023;13(1):20106. <http://dx.doi.org/10.1038/s41598-023-47219-0>.
- [27] Colin LS, Mohan PM, Pan J, Keong PL. An integrated smart contract vulnerability detection tool using multi-layer perceptron on real-time solidity smart contracts. *IEEE Access* 2024. <http://dx.doi.org/10.1109/ACCESS.2024.3364351>.
- [28] Upadhyay J, Upadhyay K, Sainju A, Poudel S, Hasan MN, Poudel K, et al. QuadraCode AI: Smart contract vulnerability detection with multimodal representation. In: *2024 33rd international conference on computer communications and networks*. 2024, p. 1–9. <http://dx.doi.org/10.1109/ICCCN61486.2024.10637655>.
- [29] Zhen Z, Zhao X, Zhang J, Wang Y, Chen H. DA-GNN: A smart contract vulnerability detection method based on dual attention graph neural network. *Comput Networks* 2024;242:110238. <http://dx.doi.org/10.1016/j.comnet.2024.110238>.
- [30] Duy PT, Khoa NH, Quyen NH, Trinh LC, Kien VT, Hoang TM, et al. Vulnsense: efficient vulnerability detection in ethereum smart contracts by multimodal learning with graph neural network and language model. *Int J Inf Secur* 2025;24(1):48. <http://dx.doi.org/10.1007/s10207-024-00965-2>.
- [31] Tong V, Dao C, Tran HA, Tran TX, Souih S. Enhancing BERT-based language model for multi-label vulnerability detection of smart contract in blockchain. *J Netw Syst Manag* 2024;32(3):63. <http://dx.doi.org/10.1007/s10922-024-09832-w>.
- [32] Ma C, Liu S, Xu G. HGAT: smart contract vulnerability detection method based on hierarchical graph attention network. *J Cloud Comput* 2023;12(1):93. <http://dx.doi.org/10.1186/s13677-023-00459-x>.
- [33] Ortner M, Eskandari S. Smart Contract Sanctuary. Retrieved from <https://github.com/tintinweb/smart-contract-sanctuary>.
- [34] Norouzi M, Fleet DJ, Salakhutdinov RR. Hamming distance metric learning. *Adv Neural Inf Process Syst* 2012;25.

- [35] Wang W, Li G, Ma B, Xia X, Jin Z. Detecting code clones with graph neural network and flow-augmented abstract syntax tree. In: 2020 IEEE 27th international conference on software analysis, evolution and reengineering. 2020, <http://dx.doi.org/10.1109/saner48275.2020.9054857>.
- [36] Liu R, Xing P, Deng Z, Li A, Guan C, Yu H. Federated graph neural networks: Overview, techniques and challenges. 2022, arXiv preprint [arXiv:220207256](https://arxiv.org/abs/220207256).
- [37] He C, Balasubramanian K, Ceyani E, Yang C, Xie H, Sun L, et al. Fedgraphnn: A federated learning benchmark system for graph neural networks. In: *ICLR 2021 workshop on distributed and private machine learning*. 2021.
- [38] <https://huggingface.co/datasets/mwritescode/sliether-audited-smart-contracts>.
- [39] Wang M, Xie Z, Wen X, Li J, Zhou K. Ethereum smart contract vulnerability detection model based on triplet loss and BiLSTM. *Electron* 2023;12(10):2327. <http://dx.doi.org/10.3390/electronics12102327>.
- [40] Ullah F, Naeem H, Naeem MR, Jabbar S, Khalid S, Al-Turjman F, et al. Detection of clone scammers in android markets using iot-based edge computing. *Trans Emerg Telecommun Technol* 2022;33(6):e3791. <http://dx.doi.org/10.1002/ett.3791>.
- [41] Zhang Z, Li D, Jiang K, Li Y, Wang J, Meng X. Propagation-based code clone analysis for detecting smart contract vulnerability. *IEICE Trans Inf Syst* 2024. <http://dx.doi.org/10.1587/transinf.2024EDL8079>.